

Importing & Managing State

Bring brownfield resources under Terraform, do surgery on state, and detect configuration drift.

Brownfield

terraform import

Generate code

State surgery & drift

Lab 08

Today's roadmap



1 - Brownfield & import

Sync existing resources into Terraform state.



2 - Generate vs. author

aztfexport and when to trust generated code.



3 - State surgery & drift

mv / rm / show and `plan -refresh-only`.



4 - Lab 08

Import a Key Vault, add RBAC & a secret.

01

Terraform & Existing Infra

Greenfield is easy. The real world is brownfield.

Existing infrastructure

Resources already exist, created by hand or another tool. To manage them, state must be **synced with reality**.

- Author (or generate) the matching HCL
- Populate state so Terraform "adopts" the resource
- Then the normal plan/apply flow takes over

1 · Write HCL

matching the real resource



2 · Import to state

terraform import



3 · plan = no changes

code now matches reality

terraform import

Bind an existing resource to a Terraform address in state.

- **address** — a valid resource address in your code
- **id** — the provider-specific resource ID
- Import fills state; it does **not** write HCL for you

BASH · IMPERATIVE

```
terraform import \  
  azurerm_resource_group.rg \  
  "/subscriptions/xxxx/resourceGroups/my-rg"
```

HCL · DECLARATIVE (TF 1.5+)

```
import {  
  to = azurerm_resource_group.rg  
  id = "/subscriptions/xxxx/resourceGroups/my-rg"  
}
```

💡 The `import { }` block is versioned in Git and runs through `plan` — prefer it for real projects.

Generating Terraform code

✓ Author by hand

- Only the properties you care about
- Clean, minimal, readable code
- You understand every line

▲ Generate (aztfexport)

- Fast for large scopes
- Emits **every** property — verbose
- Complex resources → walls of code

💡 **Azure Export for Terraform** can export one resource, a group, or a whole subscription — great for read-only inspection, use carefully for production code.

02

State Surgery & Drift

Occasionally you operate on state directly — and detect when reality wanders.

Operating on state



state show

Inspect a resource's attributes as stored in state.



state mv

Rename or move a resource (e.g. into a module) without destroying it.



state rm

Forget a resource — Terraform stops managing it, but it stays in Azure.

⚠ Powerful and manual. Always work on a backed-up, locked remote state.

Configuration drift

Drift = someone changed a resource outside Terraform. Idempotence is how IaC fixes it.

- `plan` reveals drift as proposed changes
- `plan -refresh-only` updates state to match reality
- In CI/CD, a drift check becomes a quality gate

BASH

```
# detect drift without proposing edits  
terraform plan -refresh-only  
  
# re-apply desired state to correct drift  
terraform apply
```

 LIVE DEMO

Import a resource group

Use `terraform import` (and `aztfexport`) to adopt an existing resource group, then plan to confirm no changes.

Demo · 08 **Provider** · azurerm

👤 LAB 08 · 30-40 MIN

Import a Key Vault

Create a Key Vault by hand, import it into your project, then manage a secret and an RBAC role assignment with `depends_on`.

You'll practise: `import` · `azurerm_client_config` · role assignment · `depends_on`

Key takeaways

✓ Import adopts brownfield

Write HCL, import, confirm no-change plan.

✓ Generate carefully

Great for inspection, verbose for prod.

✓ Drift is caught by plan

Idempotence keeps infra honest.

✓ Next up

Session 09 — modules.



Questions?

Up next — [Session 09: Terraform Modules.](#)